Marius Bancila's Blog
About code. Mostly on C++

Composite objects and C++26 the std::polymorphic type

October 18, 2026 by Marius Bancila

In a previous article, I discussed the C++ `std::indirect` type and how it helps implementing the PImpl idiom. However, the `std::indirect` type was inducted in the standard alongside another type called `std::polymorphic`. This helps implementing composite classes, where composite means a class composed of member objects in an has-a relation, where the composite own the components which are part of its values. Examples include strategy objects, AST/expression-tree nodes, shapes-style hierarchies, plugin-configured behavior – and you want the owning class to remain a regular, copyable value. In this article, we'll look at how `std::polymorphic` can be used for this purpose.

The need for std::polymorphic

Let us consider the following class hierarchy:

```
struct Event {
  virtual std::string describe() const = 0;
};
struct TemperatureReadingEvent : public Event {
  std::string describe() const override { }
};
struct SmokeDetectedEvent : public Event {
  std::string describe() const override { }
};
struct CarbonMonoxideDetectedEvent : public Event {
  std::string describe() const override { }
```

```
};

class EventLog {
  ??? events_;
}
```

We have various types of events from various sensors in a smart house and we want to store them in an event log. How do we do that? There are various options:

- `Event event` – slices because a `TemperatureReadingEvent` assigned into it loses its temperature specific part. Doesn't even compile here since `Event` is abstract.
- `unique_ptr<Event>` – preserves the dynamic type, but now your class is move-only (or you hand-write copying), `const` doesn't propagate, and null lurks.
- `variant<Event, TemperatureReadingEvent>` – exhibits value semantics, but maintaince a closed set: adding a `CarbonMonoxideDetectedEvent` means touching the variant and every visitor.
- `unique_ptr<Event>` and a virtual `clone()` function – the traditional "fix" for copying, which pushes boilerplate into every class in the hierarchy.

Let's try to implement the last option in the list. We start with the `Event` class:

```
#include <iostream>
#include <print>
#include <chrono>
#include <memory>
#include <string>
#include <vector>

using Clock = std::chrono::system_clock;
using TimePoint = Clock::time_point;

class Event
{
public:
   virtual ~Event() = default;

   TimePoint when() const { return ts_; }
```

```
    virtual std::string describe() const = 0;
    virtual void apply(class HomeStatus& home) const = 0;

    virtual std::unique_ptr<Event> clone() const = 0;

protected:
    TimePoint ts_ = Clock::now();

    // protect copying/assigning through base pointers fr
    Event() = default;
    Event(const Event&) = default;
    Event& operator=(const Event&) = default;
};
```

We can define an overall status of the house readings derived from event as follows:

```
class HomeStatus
{
public:
    double lastTemperature = 0.0;
    bool   smokeAlarm = false;
    bool   coAlarm = false;
    int    alertCount = 0;

    bool evacuationNeeded() const { return smokeAlarm ||
};
```

Then, we can define the actual event classes, such as the following:

```
class TemperatureReadingEvent : public Event
{
    std::string room_;
    double celsius_;
public:
    TemperatureReadingEvent(std::string room, double cels
        : room_(std::move(room)), celsius_(celsius) {}

    double celsius() const { return celsius_; }

    std::string describe() const override {
        return std::format("[{}] temperature {} C", room_,
    }
    void apply(HomeStatus& home) const override {
```

```
        home.lastTemperature = celsius_;
    }

    std::unique_ptr<Event> clone() const override {
        return std::make_unique<TemperatureReadingEvent>(*
    }
};

class SmokeDetectedEvent : public Event
{
    std::string room_;
    double density_;
public:
    SmokeDetectedEvent(std::string room, double density)
        : room_(std::move(room)), density_(density) {}

    std::string describe() const override {
        return std::format("[{}] SMOKE detected, density =
    }
    void apply(HomeStatus& home) const override {
        home.smokeAlarm = true;
        ++home.alertCount;
    }

    std::unique_ptr<Event> clone() const override {
        return std::make_unique<SmokeDetectedEvent>(*this)
    }
};

class CarbonMonoxideDetectedEvent : public Event
{
    std::string room_;
    int ppm_;
public:
    CarbonMonoxideDetectedEvent(std::string room, int ppm
        : room_(std::move(room)), ppm_(ppm) {}

    std::string describe() const override {
        return std::format("[{}] CARBON MONOXIDE {} ppm",
    }

    void apply(HomeStatus& home) const override {
        home.coAlarm = true;
        ++home.alertCount;
    }
```

```
    std::unique_ptr<Event> clone() const override {
        return std::make_unique<CarbonMonoxideDetectedEver
    }
};
```

The event log that is composed of sensor reading events may look as defined below:

```
class EventLog {
    std::vector<std::unique_ptr<Event>> events_;
public:
    EventLog() = default;

    // The Rule of Five because unique_ptr members delete
    EventLog(const EventLog& other) {
        events_.reserve(other.events_.size());
        for (const auto& e : other.events_)
            events_.push_back(e->clone());
    }

    EventLog& operator=(const EventLog& other) {
        if (this != &other) {
            std::vector<std::unique_ptr<Event>> tmp;    //
            tmp.reserve(other.events_.size());          //
            for (const auto& e : other.events_)
                tmp.push_back(e->clone());
            events_ = std::move(tmp);
        }
        return *this;
    }

    EventLog(EventLog&&) noexcept = default;
    EventLog& operator=(EventLog&&) noexcept = default;
    ~EventLog() = default;

    template <typename E, typename... Args>
    void record(Args&&... args) {
        events_.push_back(std::make_unique<E>(std::forward
    }

    HomeStatus replay() const {
        HomeStatus home;
        for (const auto& e : events_)
            e->apply(home);
```

```
        return home;
    }

    void print() const {
        for (const auto& e : events_)
            std::println("  {}", e->describe());
    }

    std::size_t size() const { return events_.size(); }
};
```

Putting it all together, we can use these classes as shown in the following snippet:

```
int main()
{
    EventLog live;
    live.record<TemperatureReadingEvent>("kitchen", 21.5
    live.record<TemperatureReadingEvent>("kitchen", 48.0
    live.record<CarbonMonoxideDetectedEvent>("garage", 12

    std::println("Live log has {} events", live.size());
    live.print();

    HomeStatus now = live.replay();
    std::println("Current home status: lastTemp={}, smoke
        now.lastTemperature,
        now.smokeAlarm,
        now.coAlarm,
        now.alertCount,
        now.evacuationNeeded());
}
```

There are a series of problems with this implementation:

- The owner loses copyability. `unique_ptr` is move-only, so the composite's copy operations are deleted. Getting them back means hand-writing a copy constructor and copy assignment that clone every component, which means the Rule of Zero is defeated.
- The clone() function affects the whole hierarchy, as every derived class must implement it. But on the other hand, the compiler can't enforce it, which means a forgotten override doesn't fail to build; it silently produces sliced copies.

- Infrastructure leaks into the interface. The function `clone()` says nothing about the type. It is a copying workaround sitting in the public API next to the methods that actually define the type.
- `const` doesn't propagate. Through a `const unique_ptr`, the pointee is still mutable (because `const` protects the pointer, not the component).
- null is representable. Nothing prevents a null pointer from entering the container or member, so every dereference carries an implicit precondition the type system doesn't express.

Enter std::polymorphic

Using the C++26 type `std::polymorphic`, we no longer need to use a cloning mechanism to allow components to behave polymorphically.

For the given example, we can simplify the `Event` class to the following:

```
class Event
{
public:
   virtual ~Event() = default;

   TimePoint when() const { return ts_; }
   virtual std::string describe() const = 0;
   virtual void apply(class HomeStatus& home) const = 0;
protected:
   TimePoint ts_ = Clock::now();
};
```

Consequently, the derived classes change as shown here:

```
class TemperatureReadingEvent : public Event
{
   std::string room_;
   double celsius_;
public:
   TemperatureReadingEvent(std::string room, double cels
      : room_(std::move(room)), celsius_(celsius) {}

   double celsius() const { return celsius_; }

   std::string describe() const override {
```

```
        return std::format("[{}] temperature {} C", room_,
    }

    void apply(HomeStatus& home) const override {
        home.lastTemperature = celsius_;
    }
};

class SmokeDetectedEvent : public Event
{
    std::string room_;
    double density_;
public:
    SmokeDetectedEvent(std::string room, double density)
        : room_(std::move(room)), density_(density) {}

    std::string describe() const override {
        return std::format("[{}] SMOKE detected, density
    }

    void apply(HomeStatus& home) const override {
        home.smokeAlarm = true;
        ++home.alertCount;
    }
};

class CarbonMonoxideDetectedEvent : public Event
{
    std::string room_;
    int ppm_;
public:
    CarbonMonoxideDetectedEvent(std::string room, int ppr
        : room_(std::move(room)), ppm_(ppm) {}

    std::string describe() const override {
        return std::format("[{}] CARBON MONOXIDE {} ppm",
    }
    void apply(HomeStatus& home) const override {
        home.coAlarm = true;
        ++home.alertCount;
    }
};
```

The `EventLog` class no longer need to adhere to the Rule of Five and implement all the special member functions. In fact, it can follow the Rule of

Zero and implement none.

```
class EventLog {
   std::vector<std::polymorphic<Event>> events_;
public:
   template <typename E, typename... Args>
   void record(Args&&... args) {
      events_.emplace_back(std::in_place_type<E>, std::f
   }

   HomeStatus replay() const {
      HomeStatus home;
      for (const auto& e : events_)
         e->apply(home);
      return home;
   }

   void print() const {
      for (const auto& e : events_)
         std::cout << "  " << e->describe() << "\n";
   }

   std::size_t size() const { return events_.size(); }
};
```

The client code does not change. The `main()` function from the previous section does not change. However, the entire code has become simpler. `std::polymorphic` solves a series of issues:

- The owner is copyable again: `std::polymorphic` is itself copyable, performing a correct deep copy of the owned object. The composite's copy operations are compiler-generated, which means they follow the Rule of Zero.
- No `clone()` protocol: the copy operation is type-erased and captured at construction, when the concrete type is statically known. The hierarchy needs no `clone()` function, and there's nothing for a derived class to forget.
- Clean interfaces: with no `clone()` to declare, the base class contains only domain methods.
- `const` propagates: through a `const std::polymorphic`, access to the owned object is const. Deep const is enforced by the type system, not by convention.

- Never null: a `std::polymorphic` object always owns a value; there is no null state to construct. The only empty state is moved-from, explicitly detectable via `valueless_after_move()`, so every dereference elsewhere is valid by invariant.

In short, the `std::polymorphic` component behaves like a regular member again: copyable, deeply const, always present, while keeping its dynamic type.

You may ask, when you copy a `std::polymorphic<Event>` that holds a `TemperatureReadingEvent`, how does it know to copy a `TemperatureReadingEvent` without a virtual `clone()`? The answer is type erasure of the copy operation, captured at construction time.

Conceptually, `std::polymorphic<Event>` stores its object behind a small internal control structure. When you construct it with `std::in_place_type<TemperatureReadingEvent>`, the constructor, which at that moment statically knows the type is `TemperatureReadingEvent`, instantiates and stores operations equivalent to the following:

```
copy    = [](const Event& e) { return new TemperatureRea
destroy = [](Event* e)       { delete static_cast<Event
```

Later, `std::polymorphic`'s copy constructor just invokes the stored operation. It's the same technique `std::function` and `std::any` use, applied to the copy constructor. This also means `Event` doesn't even need a virtual destructor for `std::polymorphic` to destroy it correctly. The erased destroy operation already knows the concrete type. That doesn't mean you should remove the the virtual destructor, as other code may still delete through base pointers.

The rest of the interface mirrors the `std::indirect` type:

- `operator*` / `operator->` give you `Event&` / `Event*`, const-propagated (through a `const std::polymorphic` you get `const Event&`).
- Never null, except the moved-from state, queryable with `valueless_after_move()`.
- Allocator-aware, and unlike `std::indirect`, implementations are permitted a small-buffer optimization, storing small derived objects inline without heap allocation.

- There's no way to ask "which derived type is inside?". It doesn't feature a `type()` function like `std::any` has. If you need that, use `dynamic_cast` on `*p` like you would with any base reference. The design intent is that you interact through the base interface.

Conclusion

Use `std::polymorphic` when a class owns a component whose dynamic type varies at runtime and that component is part of the owner's value: copying the owner should copy it, `const` should propagate to it, and it should never be absent. Typical cases are strategy members, heterogeneous collections like event logs, and recursive structures like expression trees.

If the type is fixed, use `std::indirect`.

If the set of alternatives is closed, use `std::variant`.

If the object is a shared identity rather than a value, stick with smart pointers.

See also

Here is a list of additional readings on the subject:

- std::polymorphic
- Copying Polymorphic Types: How C++26 simplifies value semantics
- C++26: std::polymorphic

C++

C++, C++26

‹ cppdashboard.dev – The aggregator of what matters in C++ right now

Leave a Reply